

Supervised Machine Learning: Regression and Classification



Summary

Supervised learning

Learns from being given "right answers".

Regression

- Predict a number
- Infinitely many possible outputs

Classification

- Predict categories
- Small number of possible outputs

Unsupervised learning

Data only comes with inputs x, but not output labels y. Algorithm has to find "structure" in the data.

Clustering

Group similar data points together

Dimensionality reduction

Compress data using fewer numbers.

Anomaly Detection

Find unusual data points.

Machine Learning

Field of study that gives computers the ability to learn without being explicitly programmed."

— Arthur Samuel (1959)

Machine learning algorithms

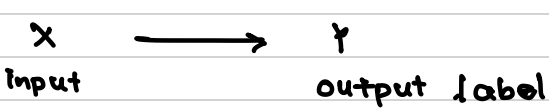
→ Supervised learning (course 1, 2) — rapid advancements — used most in real-world applications

→ unsupervised learning (course 3)

→ Recommender systems

→ Reinforcement learning

Supervised Machine learning

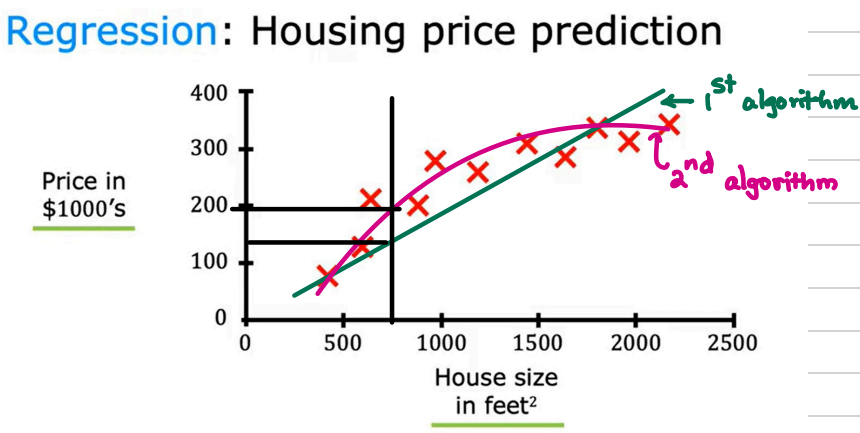


Learns from being given "right answers"

By seeing the correct pair of input x and desired output label y that the learning algorithm eventually learns to take just the input alone without the output label and gives a reasonably accurate prediction or guess of the output.

Input (X)	output (Y)	Application
email	spam ? (0/1)	Spam filtering
audio	text transcripts	Speech recognition
English	Spanish	machine translation
ad, user info	click ? (0/1)	online advertising
image, radar info	position of other cars	Self driving car
image of phone	defect ? (0/1)	visual inspection

Train the model with inputs and their right answers. After the model has learned from these inputs and outputs, they can take a brand new input and try to produce corresponding output.



How could we decide to fit for a straight line or curve or any other fw ?

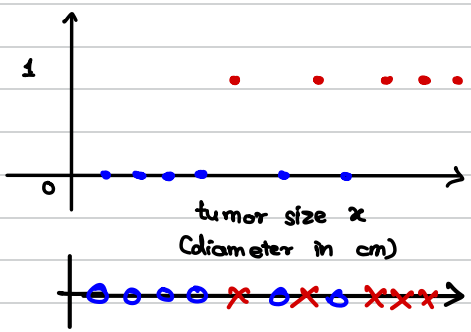
Regression
→ Predict a number
→ Infinitely many possible outputs

Classification : Breast cancer detection


lump

- malignant → cancerous or dangerous
- benign → isn't cancerous or isn't dangerous

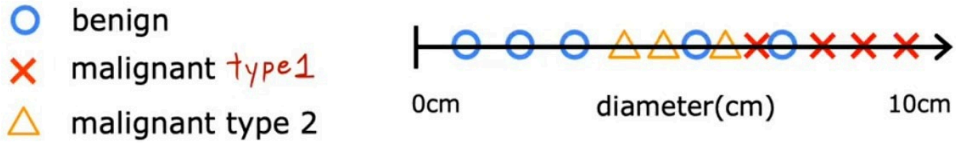
Size	diagnosis
2	0
5	1
1	0
7	1
:	
:	



Only a small no. of outputs or categories. In this case two possible outputs. → Benign or → Malignant.

} differences from regression

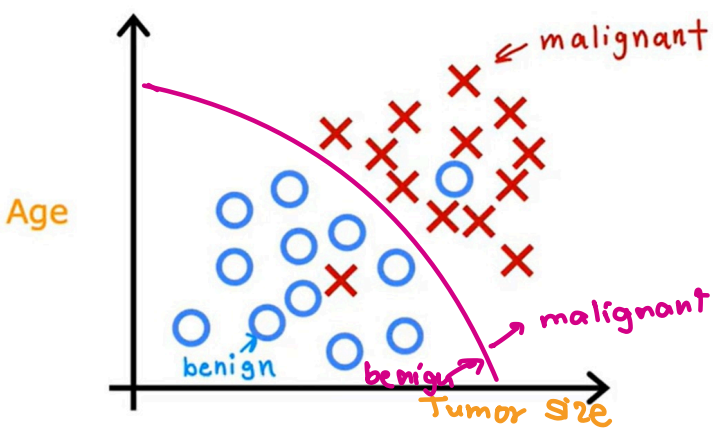
As there are only two possible outputs we can represent data on a line.



Classification → class or categories
Predict categories (Ex :- cat or dog, benign or malignant

small number of possible outputs like 0, 1, 2/not like 1.7 , 2.5

Two or more inputs



Age and size are the two inputs

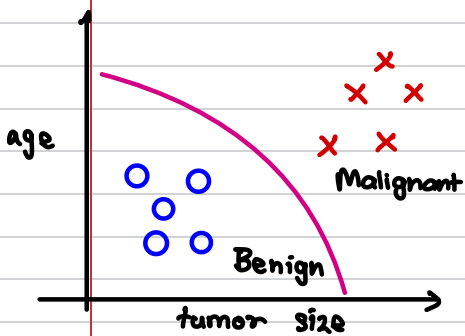
The learning algorithm should find some boundary that seperates out the malignant tumors from the benign ones.

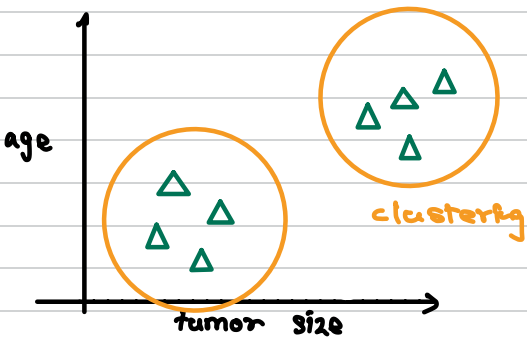
Can use more input data like,
- the thickness of the tumor clump
- Uniformity of the cell size
- Uniformity of the cell shape

Unsupervised learning

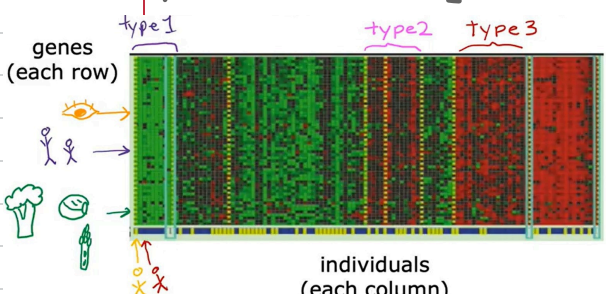
Supervised learning learn from data labeled with the "right answers"

Unsupervised learning find something interesting in unlabeled data.





2) DNA microarray

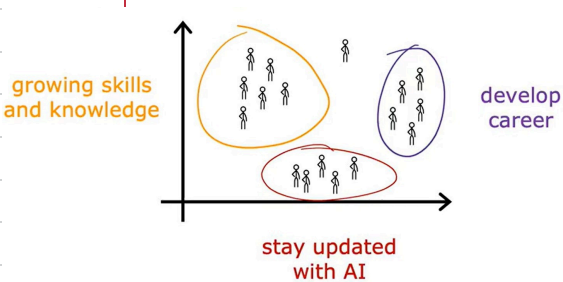


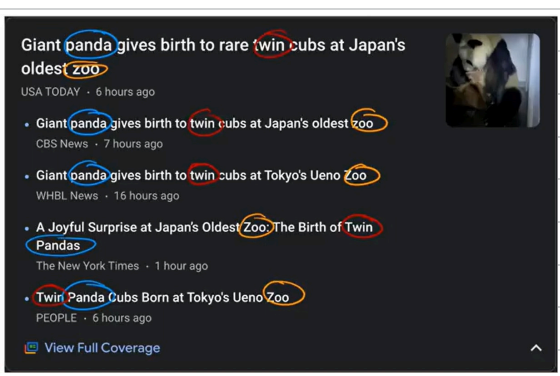
The unsupervised learning algorithm, might decide that the data can be assigned to two different groups or two different clusters.

Eg :- clustering

Ex:- i) google news

3) grouping customers







Numbers

Regression model predict numbers

Terminology

Training set – Data used to train the model

x: size in feet²	y: price in $1000's
(1) 2104	400
(2) 1416	232
(3) 1534	315
(4) 852	178
⋮	⋮
(47) 3210	870

Notation :-

x = "input" variable
feature

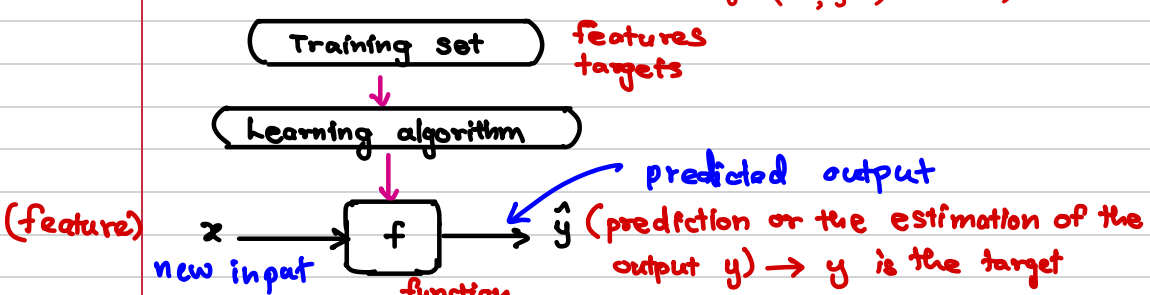
y = "output" variable
"target" variable

m = Total no. of training examples (47)

(x, y) = Single training example

$(x^{(i)}, y^{(i)})$ = i^{th} training example

Eg:- $(x^{(1)}, y^{(1)}) = (2104, 400)$



How to represent f ?

$f_{w,b}(x) = wx + b$

$f(x)$

Optional lab

```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('./deeplearning.mplstyle')
# x_train is the input variable (size in 1000 square feet)
# y_train is the target (price in 1000s of dollars)
x_train = np.array([1.0, 2.0])
y_train = np.array([300.0, 500.0])
print(f"x_train = {x_train}")
print(f"y_train = {y_train}")
# m is the number of training examples
m = len(x_train)
print(f"Number of training examples is: {m}")
i = 1 # Change this to 1 to see (x^1, y^1)

x_i = x_train[i]
y_i = y_train[i]
print(f"(x^({i}), y^({i})) = ({x_i}, {y_i})")
# Plot the data points
plt.scatter(x_train, y_train, marker='x', c='r')
# Set the title
plt.title("Housing Prices")
# Set the y-axis label
plt.ylabel('Price (in 1000s of dollars)')
# Set the x-axis label
plt.xlabel('Size (1000 sqft)')
plt.show()
w = 100
b = 100
print(f"w: {w}")
print(f"b: {b}")

def compute_model_output(x, w, b):
    """
    Computes the prediction of a linear model
    Args:
      x (ndarray (m,)): Data, m examples
      w,b (scalar)    : model parameters
    Returns
      f_wb (ndarray (m,)): model prediction
    """
    m = x.shape[0]
    f_wb = np.zeros(m)
    for i in range(m):
        f_wb[i] = w * x[i] + b

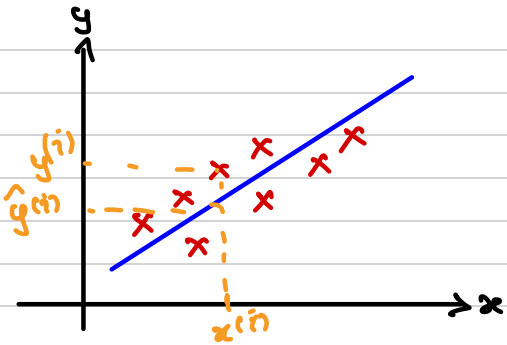
    return f_wb

tmp_f_wb = compute_model_output(x_train, w, b,)

# Plot our model prediction
plt.plot(x_train, tmp_f_wb, c='b',label='Our Prediction')

# Plot the data points
plt.scatter(x_train, y_train, marker='x', c='r',label='Actual Values')

# Set the title
plt.title("Housing Prices")
# Set the y-axis label
plt.ylabel('Price (in 1000s of dollars)')
# Set the x-axis label
plt.xlabel('Size (1000 sqft)')
plt.legend()
plt.show()
```

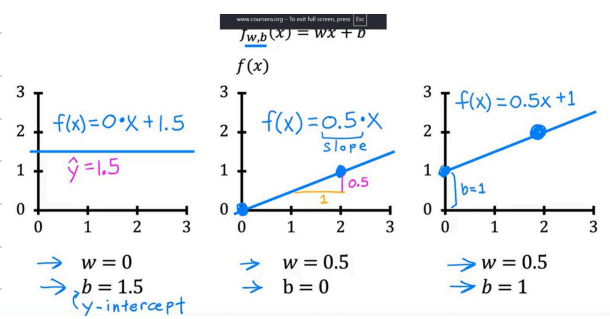


Linear regression with one variable .that means single feature x (like size)

Univariate linear regression

Model : $f_{w,b}(x) = wx + b$

w, b : parameters
coefficients
weights



$\hat{y}^{(i)} = f_{w,b}(x^{(i)})$

$f_{w,b}(x^{(i)}) = wx^{(i)} + b$

cost function : squared error cost function

average error

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$ — error

m = number of training examples

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2$$

find w, b :-

$\hat{y}^{(i)}$ is so close to $y^{(i)}$ for all $(x^{(i)}, y^{(i)})$

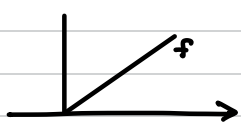
Simplified

model :- $f_{w,b}(x) = wx + b$ | $f_w(x) = wx$

parameters :- w, b



w



cost function :-

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2$$

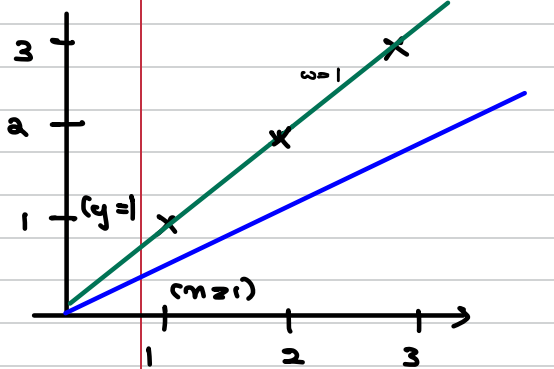
$$J(w) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_w(x^{(i)}) - y^{(i)} \right)^2$$

goal :- $\underset{w,b}{\text{minimize}}\ J(w,b)$ | $\underset{w}{\text{minimize}}\ J(w)$

$f_w(x)$

(for fixed w, function of x)

(input)



$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}\left[f_w(x^{(i)}) - y^{(i)}\right]^2$$

$$= \frac{1}{2m}\sum_{i=1}^{m}\left[wx^{(i)} - y^{(i)}\right]^2$$

$w = 1$;

$$= \frac{1}{2m}(0^2 + 0^2 + 0^2) = 0$$

$w = 0.5$,

$$= \frac{1}{2m}(0.5^2 + 1^2 + 1.5^2)$$

$$= \frac{1}{2\times3}(0.25 + 1 + 2.25)$$

$$= \frac{1}{2\times3}(3.5) = 0.58$$

$w = 0$

$$= \frac{1}{2m}(1^2 + 2^2 + 3^2)$$

$$= \frac{1}{2\times3}(1 + 4 + 9)$$

$$= \frac{1}{2\times3} \times 14 = 2.33$$

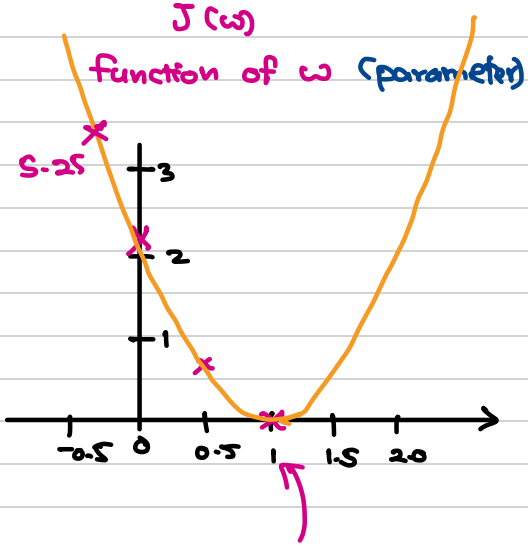
when $w = -0.5$ it becomes

5.25

$(b = 0)$

$J(w)$

function of w (parameter)

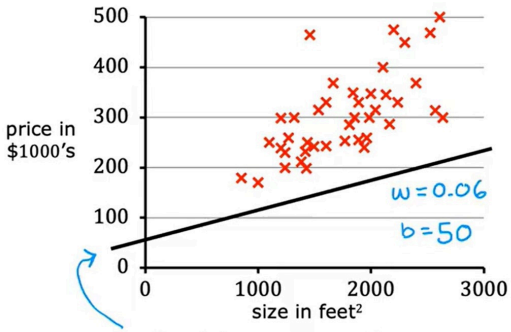


$w = 1$ $J = 0$

gives the best fit curve for data set

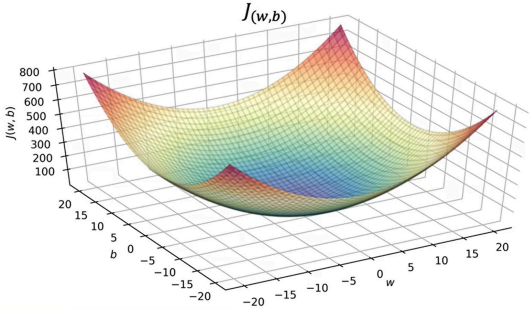
[As i think it's actually where the derivative of J is equal to zero).

$f_{w,b}$
(function of x)

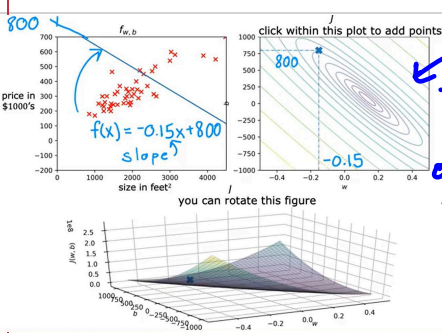


$$f_{w,b}(x) = 0.06x + 50$$

J
(function of w, b)

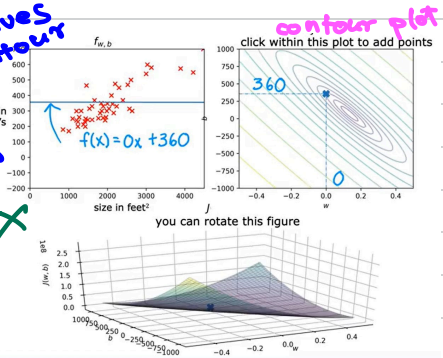


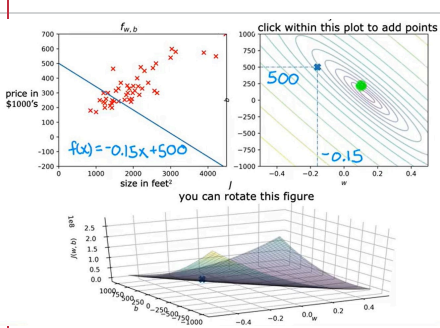
3D view plotted in matlab when w = 0.06 and b = 50.



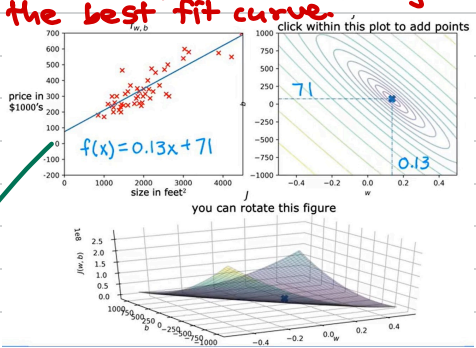
This gives the contour lines of the 3D view

contour plot





The minimum J value gives the best fit curve.



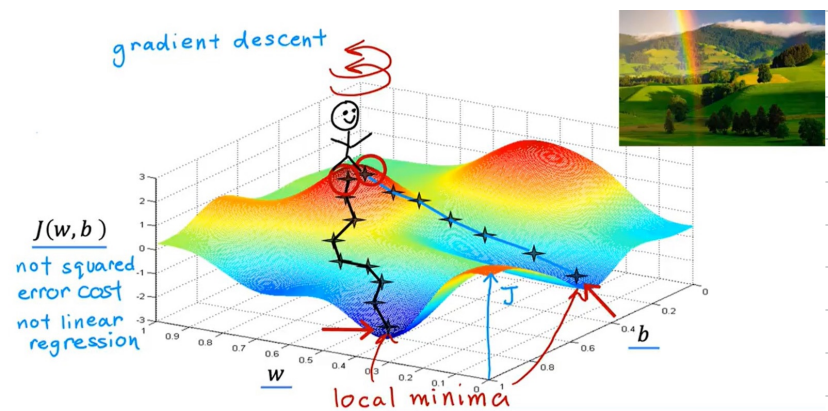
Gradient Decent

Is an algorithm that you can use to try to minimize any function, not just a cost function for linear regression.

outline:-

Start with some w, b (set $w=0, b=0$)

Keep changing w, b to reduce $J(w,b)$ until we settle at or near minimum.



Gradient descent algorithm

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

Repeat until convergence

Learning rate (it's a value between 0 & 1)

Controls how big of a step you take down-hill)

derivative

assignment operator

* Simultaneously update w and b

Simultaneous update

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

$$w = tmp_w$$

$$b = tmp_b$$

Incorrect

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$w = tmp_w$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

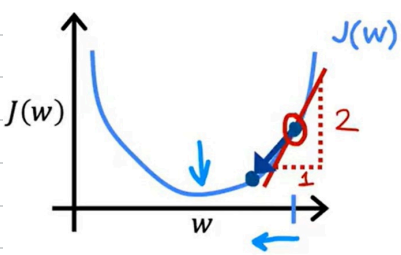
$$b = tmp_b$$

Eg:-

first think about the J cost function with one parameter w

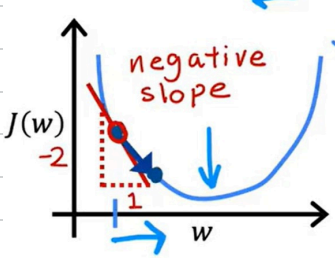
$$J(w) \Rightarrow \quad w = w - \alpha \frac{d}{dw} J(w)$$

α is always a positive number



$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

$$w = w - \alpha \cdot (\text{positive number})$$



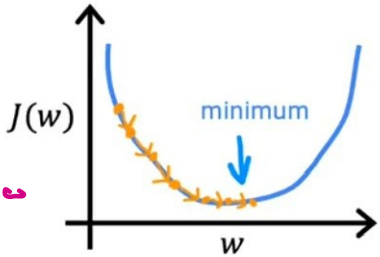
w getting a smaller number

$$\underbrace{\frac{d}{dw} J(w)}_{<0}$$

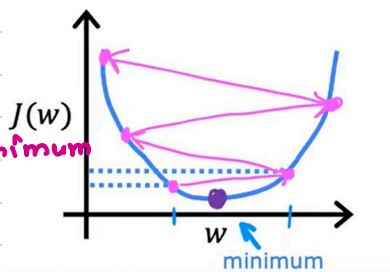
$$w = w - \alpha \cdot (\text{negative number})$$

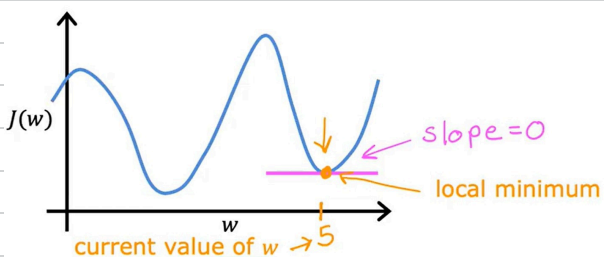
$$w = w - \alpha \frac{d}{dw} J(w)$$

If α is too small,
Gradient descent may be slow



If α is too large,
Gradient descent may:
- overshoot, never reach minimum
- Fail to converge, diverge





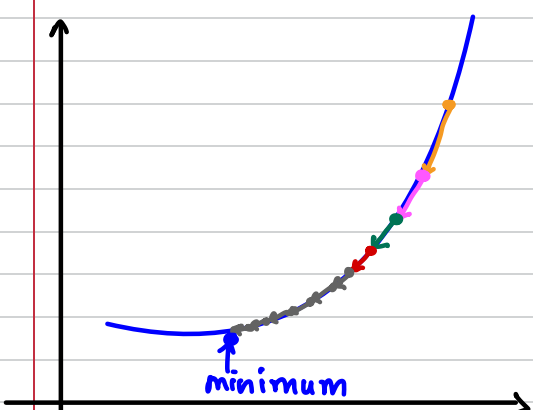
$$w = w - \alpha \frac{d}{dw} J(w)$$

if $\alpha = 0.1$

$w = w - 0.1 \times 0$

$w = w$

Can reach local minimum with fixed learning rate



$$w = w - \alpha \frac{d}{dw} J(w)$$

large

not as large

smaller

Near a local minimum,
- Derivative becomes smaller
- Update steps become smaller

Can reach minimum without decreasing learning rate (α)

Linear Regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left[f_{w,b}(x^{(i)}) - y^{(i)} \right]^2$$

Gradient descent algorithm

Repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$= w - \alpha \frac{\partial}{\partial w} \left\{ \frac{1}{2m} \sum_{i=1}^{m} \left[f_{w,b}(x^{(i)}) - y^{(i)} \right]^2 \right\}$$

$$= w - \alpha \frac{\partial}{\partial w} \left\{ \frac{1}{2m} \sum_{i=1}^{m} \left[wx^{(i)} + b - y^{(i)} \right]^2 \right\}$$

$$= w - \alpha \left\{ \frac{1}{2m} \times \sum_{i=1}^{m} \left[wx^{(i)} + b - y^{(i)} \right] x^{(i)} \right\}$$

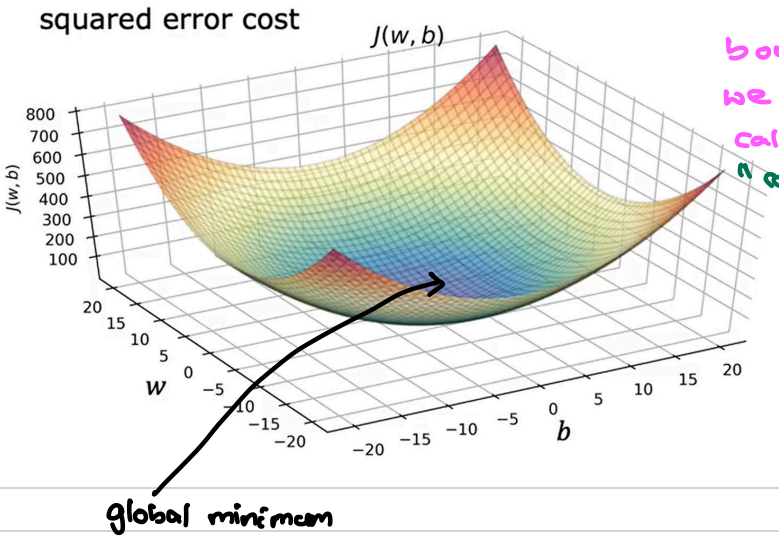
$$w = w - \frac{\alpha}{m} \sum_{i=1}^{m} \left[f_{w,b}(x^{(i)}) - y^{(i)} \right] x^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

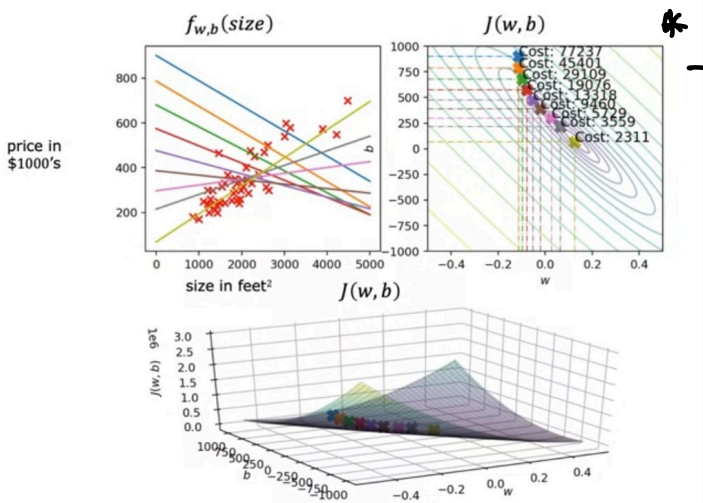
$$= b - \alpha \cdot \frac{\partial}{\partial b} \left\{ \frac{1}{2m} \sum_{i=1}^{m} \left[f_{w,b}(x^{(i)}) - y^{(i)} \right]^2 \right\}$$

$$= b - \alpha \cdot \left\{ \frac{1 \times 2}{2m} \sum_{i=1}^{m} \left[f_{w,b}(x^{(i)}) - y^{(i)} \right] \times 1 \right\}$$

$$b = b - \frac{\alpha}{m} \sum_{i=1}^{m} \left[f_{w,b}(x^{(i)}) - y^{(i)} \right]$$



bowl shape we technically call this as "convex function".



* At global minima it gets the perfect fit for the values.

Other gradient descent :-
Subsets

"Batch" gradient descent

"Batch" : Each step of gradient descent uses all the training examples.

$$\sum_{i=1}^{m} \left[f_{w,b}(x^{(i)}) - y^{(i)} \right]^2$$

Multiple features (variables)

x_1 Size in feet^2	x_2 No. of bedrooms	x_3 No. of floors	x_4 Age of home in years	Price ($) in $1000's
2104	5	1	45	460
1416	3	2	40	232 $\Rightarrow \hat{y}=2$
1534	3	2	30	315
852	2	1	36	178
⋮	⋮	⋮	⋮	⋮

Multiple features to predict price

x_j = j^{th} feature (here $j = 1, 2, 3, 4$)

n = no. of features ($n = 4$)

To indicate it's a vector → $\vec{x}^{(i)}$ = features of ith training example

$\vec{x}^{(i)} = [1416 \quad 3 \quad 2 \quad 40]$ ← Row vector

$x_j^{(i)}$ = value of feature j in i^{th} training example

$x_3^{(2)} = 2$

Model :- Previously → $f_{w,b}(x) = wx + b$

$$f_{w,b}(x) = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

example →

$$f_{w,b}(x) = 0.1x_1 + 4x_2 + 10x_3 + -2x_4 + 80$$

x_1: Size; x_2: #bedrooms; x_3: #floors; x_4: years; 80: Base price

So the general function,

$$f_{w,b}(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

$\vec{w} = [w_1 \quad w_2 \quad w_3 \dots \quad w_n]$ } parameters of the model

b is a number

$\vec{x} = [x_1 \quad x_2 \quad x_3 \dots \quad x_3]$

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

dot product of two row vectors gives a scalar And to do that should have same n for both the matrices

$$= (w_1x_1 + w_2x_3 + \dots + w_nx_n) + b$$

Multiple linear regression

Parameters and features

$\vec{w} = [w_1 \quad w_2 \quad w_3]$
b is a number
$\vec{x} = [x_1 \quad x_2 \quad x_3]$
n=3

```
w = np.array([1.0, 2.5, -3.3])
b = 4
x = np.array([10, 20, 30])
```

linear algebra : count from 1 Code : count from 0

without vectorization

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

```
f = w[0] * x[0] +
    w[1] * x[1] +
    w[2] * x[2] + b
```

But if n = 100,000 this is hard for writer also the computer

(4)

$$f_{\vec{w},b}(\vec{x}) = \left(\sum_{j=1}^{n} w_j x_j\right) + b$$

$\sum_{j=1}^{n} \rightarrow j = 1, \dots n$ 1,2,3

range(0,n) → j = 0 ... n-1

```
f = 0
for j in range(0,n):
    f = f + w[j] * x[j]
f = f + b
```

But this somewhat a higher computation -al working

(11)

Vectorization

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

```
f = np.dot(w,x) + b
```



numpy dot function is a vector-ize implementation of dot operation between two vectors. It runs much faster than the previous two methods.

Benefits of using vectorization

(1) it makes the code shorter

(2) results in running the code much faster than when vectorization is not used.

The reason is numpy dot function is able to use parallel hardware in your computor

without vectorization

```
for j in range(0,16):
    f = f + w[j] * x[j]
```

t_0 f + w[0] * x[0]

t_1 f + w[1] * x[1]

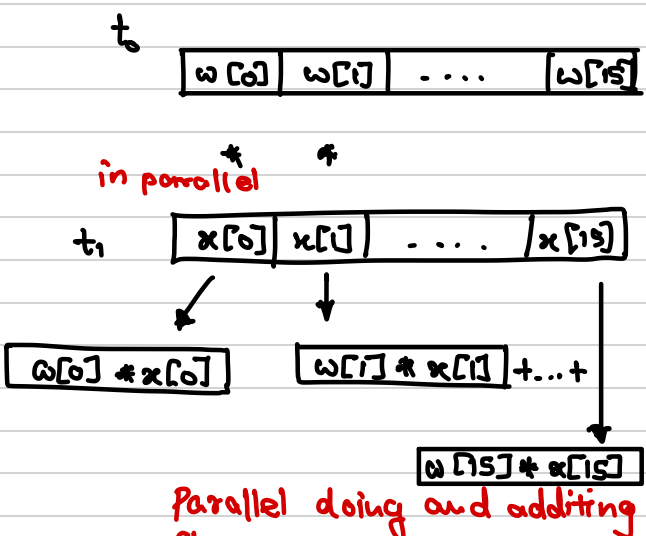
...

t_{15} f + w[15] * x[15]

One after the other

Vectorization

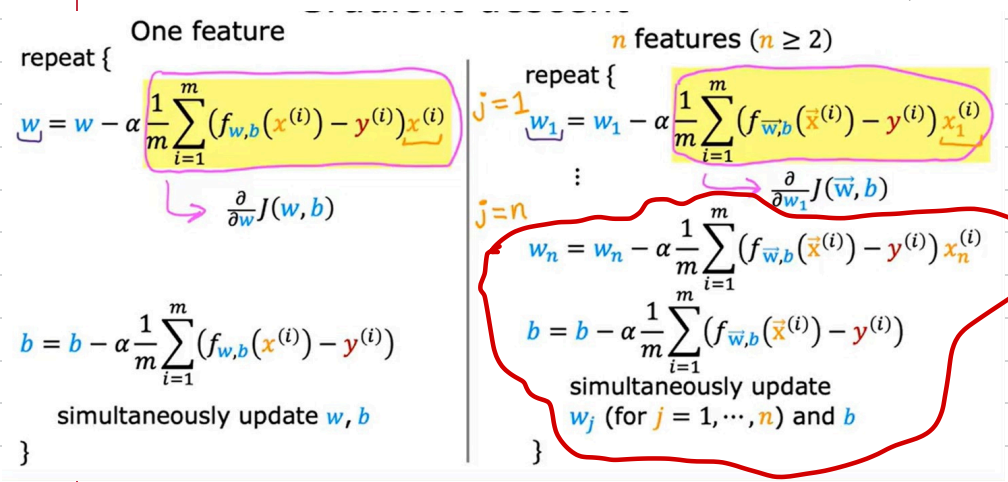
```
np.dot(w,x)
```



Parallel doing and additing it once.

	Previous Notation	Vector notation
Parameters	$w_1, \dots, w_n$ b	$\vec{w} = [w_1 \dots w_n]$ b still a number
Model	$f_{\vec{w},b}(\vec{x}) = w_1x_1 + \dots + w_nx_n + b$	$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$ ↑ dot product
cost function	$J(w_1, \dots, w_n, b)$	$J(\vec{w}, b)$

Gradient descent	Repeat {
	$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(w_1, \dots w_n, b)$
	$b = b - \alpha \frac{\partial}{\partial b} J(w_1, \dots, w_n, b)$
	}
		Repeat {
		$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$
		$b = b - \alpha \frac{\partial}{\partial b} [J(\vec{w}, b)]$



An alternative to gradient descent

→ Normal equation

* only for linear regression
* solve for w, b without iterations

Disadvantages

* Doesn't generalize to other learning algorithms
* Slower when no. of features is large ($> 10,000$)

What you need to know

- Normal equation method may be used in machine lear libraries that implement linear regression.
- Gradient descent is the recommended method for finding parameters w

Feature and parameter values

$\hat{price} = w_1 x_1 + w_2 x_2 + b$

$w_1 x_1$: size — $w_2 x_2$: # of bedrooms

x_1 : size (feet^2)
range : 300 − 2000
large

x_2 : # bedrooms
range : 0 − 5
small

House : $x_1 = 2000$, $x_2 = 5$, price = \$500 k
one training example

Size of the parameters w_1 and w_2

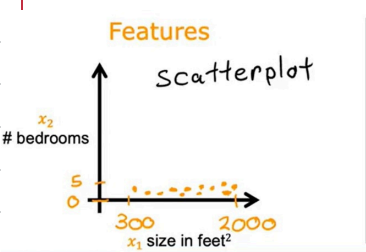
$w_1 = 50$, $w_2 = 0.1$, $b = 50$

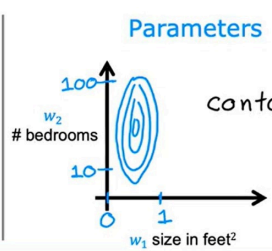
$\hat{price} = 50 * 2000 + 0.1 * 5 + 50$
$= \$ 100,050.5$ k
far away from the prediction

$w_1 = 0.1$, $w_2 = 50$, $b = 50$
small, large

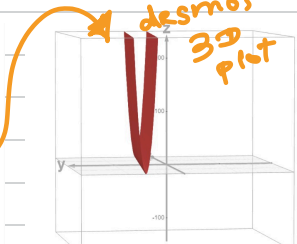
$\hat{price} = 0.1 * 2000$ k $+ 50 * 5 + 50$
$= \$500$ k
More reasonable

	size of feature (x_j)	size of parameter (w_j)
size in feet^2	← →	← →
# bedrooms	← →	← →

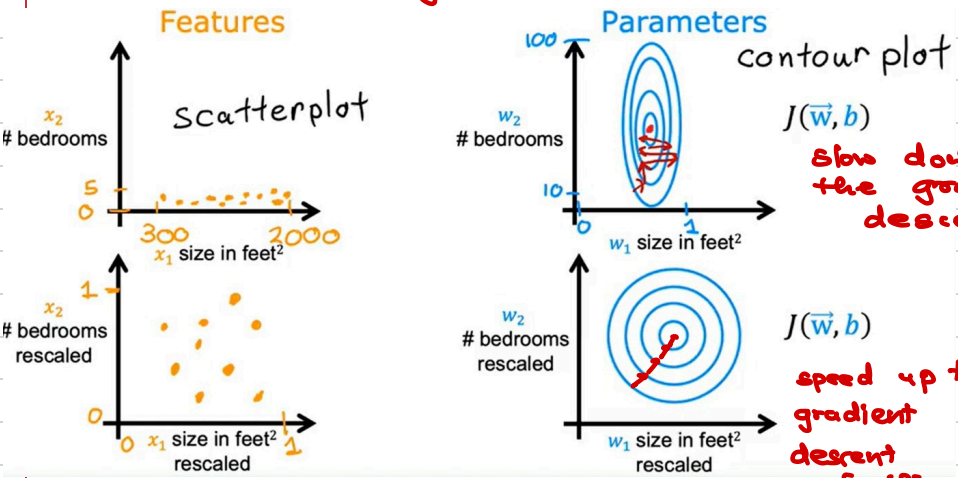




I drew in desmos 3D plot



Feature size and gradient descent



w jump back and forth to find the optimal w.

Feature rescaling

Just dividing by the maximum

$300 \leq x_1 \leq 2000$

$x_{1,\text{scaled}} = \frac{x_1}{2000}$ (highest number)

$0.15 \leq x_{1,\text{scaled}} \leq 1$

$0 \leq x_2 \leq 5$

$x_{2,\text{scaled}} = \frac{x_2}{5}$ (max)

$0 \leq x_{2,\text{scaled}} \leq 1$

Mean normalization

$\mu_1 = 600$, $\mu_2 = 2.3$

$300 \leq x_1 \leq 2000$

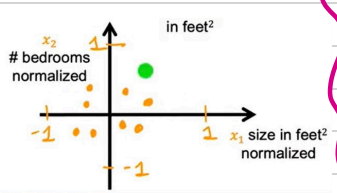
$x_1 = \frac{x_1 - \mu_1}{2000 - 300}$ (max − min)

$-0.18 \leq x_1 \leq 0.82$

$0 \leq x_2 \leq 5$

$x_2 = \frac{x_2 - \mu_2}{5 - 0}$ (max − min)

$-0.46 \leq x_2 \leq 0.54$



Z-score normalization

$300 \leq x_1 \leq 2000$

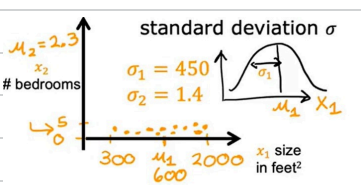
$x_1 = \frac{x_1 - \mu_1}{\sigma_1}$

$-0.67 \leq x_1 \leq 3.1$

$0 \leq x_2 \leq 5$

$x_2 = \frac{x_2 - \mu_2}{\sigma_2}$

$-1.6 \leq x_2 \leq 1.9$



aim for about $-1 \leq x_j \leq 1$ for each feature x_j

$-3 \leq x_j \leq 3$ } acceptable ranges

$-0.3 \leq x_j \leq 0.3$

Ex:- $0 \leq x_1 \leq 3$ — okay, no rescaling

$-2 \leq x_2 \leq 0.5$ — okay, no rescaling

$-100 \leq x_3 \leq 100$ — too large → rescale

$-0.001 \leq x_4 \leq 0.001$ — too small → rescale

$98.6 \leq x_5 \leq 105$ — too large → rescale

(Body temperature of patients)

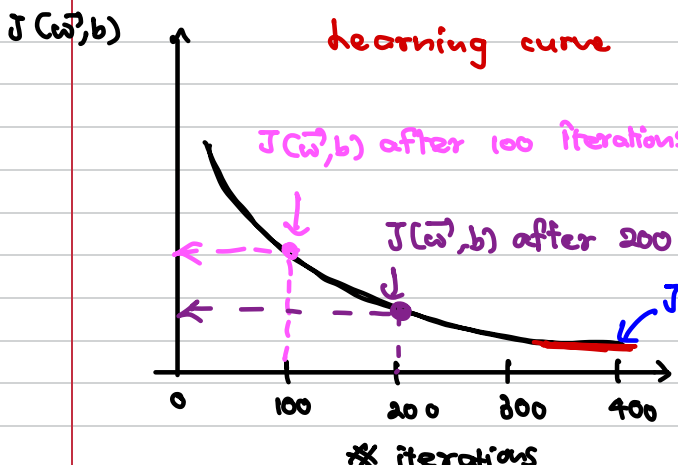
patient & body temperature

↑ large (98-100)

(5-30)

Make sure gradient descent is working correctly

objective : $\min_{\vec{w},b} J(\vec{w},b)$ $J(\vec{w},b)$ should decrease after every iteration



[If J increases after an iteration that may be α has choosen poorly, (α large) or there might be a bug in the code]

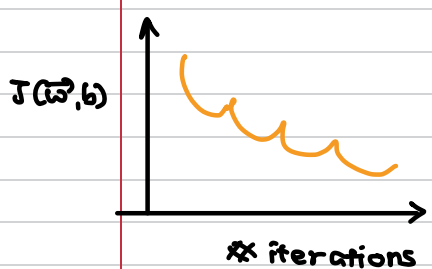
No of iterations varies a lot from application to application. Therefore we need to observe the learning curve at what iteration it might have converging ✓ as we cannot say it in advance.

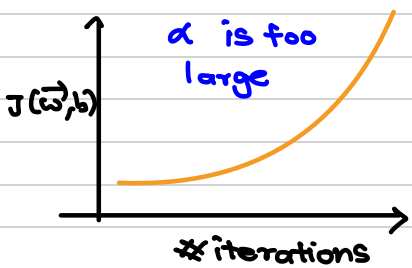
Another way of testing convergence apart from the learning curve is that Automatic convergence test.

Let ε "epsilon" be 10^{-3} (0.001)

If $J(\vec{w}, b)$ decreases by $\leq \varepsilon$ in one iteration, declare in one iteration, declare convergence
(found parameters $\vec{w}, b$ to get close to global minimum).

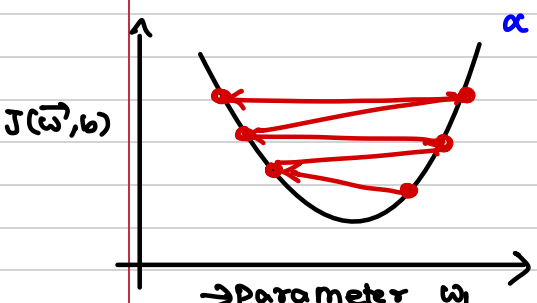
Identify problem with gradient descent

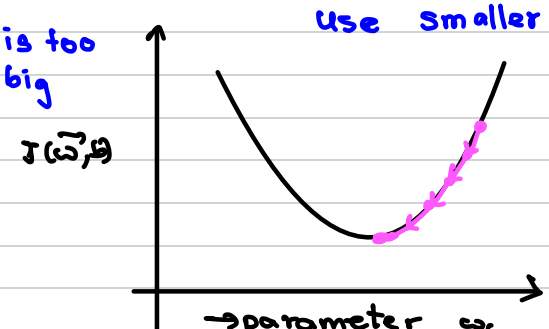




α is too large or there may be a bug in the code.

Adjust learning rate





$w_1 = w_1 + \alpha d_1$ ← This is wrong !! Because of '+' sign

use a minus sign

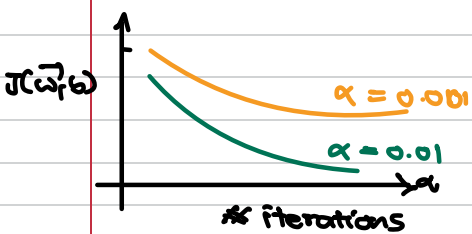
$w_1 = w_1 - \alpha d_1$!!

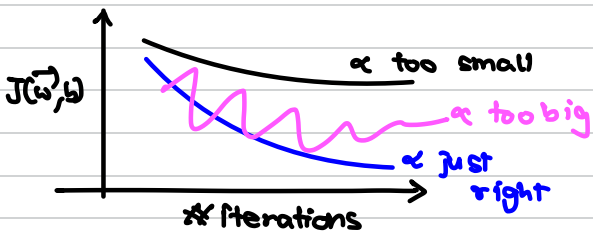
With a small enough α, $J(\vec{w}, b)$ should decrease on every iteration.

If α is too small, gradient descent takes a lot more iterations to converge.

Values of α to try:

0.001 (×3) 0.003 (≈×3) 0.01 (×3) 0.03 (≈×3) 0.1 (×3) 0.3 (≈×3) 1





Feature engineering

Feature engineering

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + b$$

x_1: frontage, x_2: depth

$$area = frontage \times depth$$

$$x_3 = x_1 x_2$$

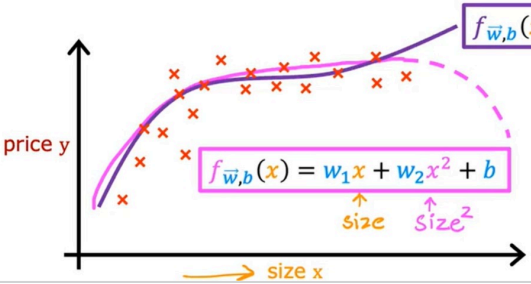
new feature

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$



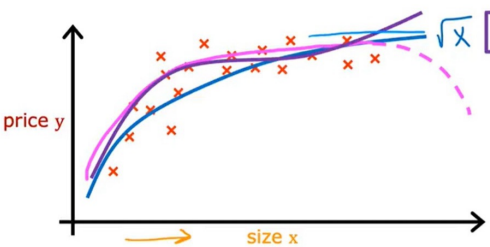
Feature engineering: Using intuition to design new features, by transforming or combining original features.

Polynomial Regression



Because of the previous error happens in gradient descent.

Choice of features



what features to use?
↳ course 2 (Advanced algorithms)

So in course 2 you will learn what are the features to include or don't include and we have a process for measuring how good these models to decide whether to include or not these features.

Classification

Question	Answer "y"	
Is this email spam?	no	yes
Is the transaction fraudulent?	no	yes
Is the tumor malignant?	no	yes

y can only be one of the two values. → "Binary classification"

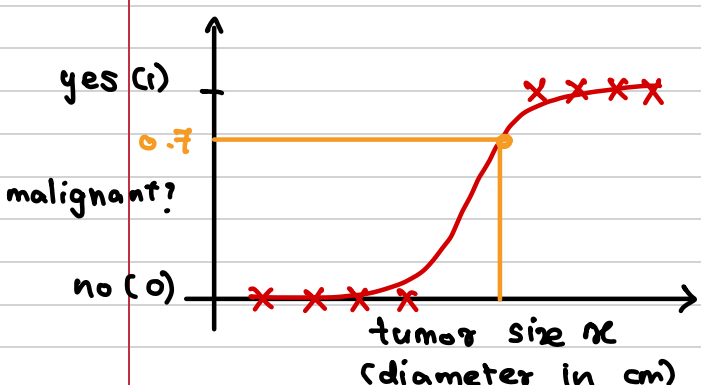
class = category

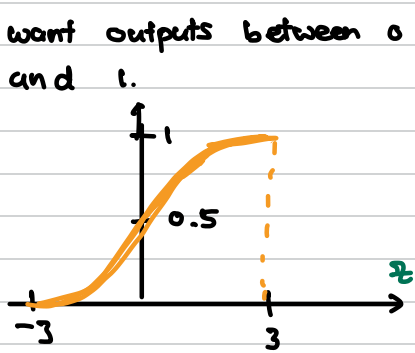
false (0) → "Negative class" → absence

True (1) → "positive class" → presence

It doesn't mean it's bad or good.

logistic regression





Sigmoid function

we can do the same thing using linear regression. But when there are some points which changes the best fit curve, then it will give false results.

logistic function outputs between 0 & 1

$$g(z) = \frac{1}{1+e^{-z}} \qquad 0 < g(z) < 1$$

$f_{\vec{w},b}(\vec{x})$

$z = \vec{w} \cdot \vec{x} + b$

$g(z) = \frac{1}{1+e^{-z}}$

"logistic regression"

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

"probability" that class is 1

$f_{\vec{w},b}(\vec{x}) = P(y=1 \mid \vec{x}; \vec{w}, b)$

Example :-

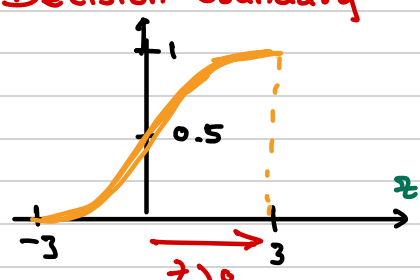
x is "tumor size"

y is 0 (not malignant) or 1 (malignant)

Probability that y is 1, given input x, parameters $\vec{w}, b$.

$f_{\vec{w},b}(\vec{x}) = 0.7$ 70% chance that y is 1.

Decision Boundary



$P(y=0) + P(y=1) = 1$

$$f_{\vec{w},b}(\vec{x}) = g(\underbrace{\vec{w} \cdot \vec{x} + b}_{z}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}} = P(y=1 \mid x; \vec{w}, b)$$

Threshold

Is $f_{\vec{w},b}(\vec{x}) \geq 0.5$? Yes : $\hat{y}=1$, No : $\hat{y}=0$

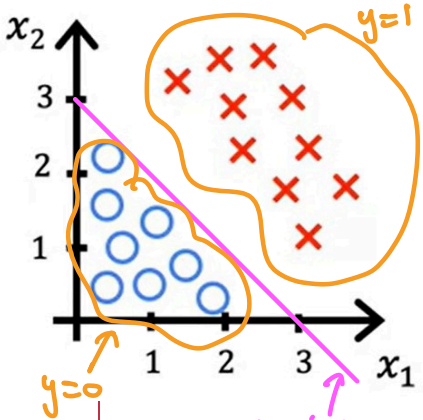
When is $f_{\vec{w},b}(\vec{x}) \geq 0.5$?

$g(z) \geq 0.5$

$z \geq 0$

$\vec{w} \cdot \vec{x} + b \geq 0$ $\quad y=1$

$\vec{w} \cdot \vec{x} + b < 0$ $\quad \hat{y}=0$

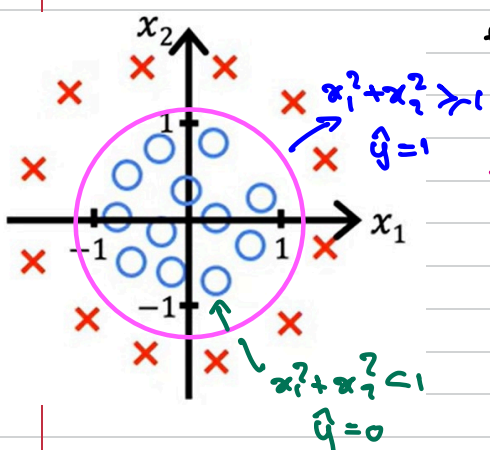


$$f_{\vec{w},b}(\vec{x}) = g(z)$$
$$= g(w_1x_1 + w_2x_2 + b)$$
$$1 \quad 1 \quad -3$$

Decision boundary

$$z = \vec{w}\cdot\vec{x} + b = 0$$
$$z = x_1 + x_2 - 3 = 0$$
$$x_1 + x_2 = 3$$

Non-linear decision boundaries

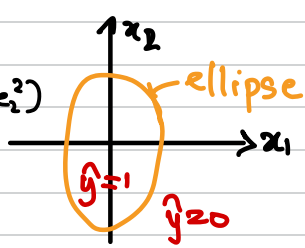


$$f_{\vec{w},b}(\vec{x}) = g(z)$$
$$= g(w_1x_1^2 + w_2x_2^2 + b)$$
$$1 \quad 1 \quad -1$$

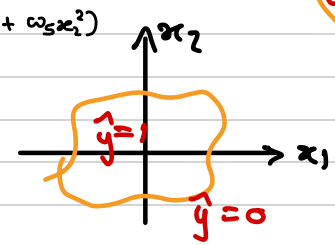
Decision boundary

$$z = x_1^2 + x_2^2 - 1 = 0$$
$$x_1^2 + x_2^2 = 1$$

$$f_{\vec{w},b}(\vec{x}) = g(z) = g(w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2)$$



$$f_{\vec{w},b}(\vec{x}) = g(z) = g(w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2)$$



	tumor size (cm) x_1	...	Patient's age x_n	malignant y
$i=1$	10		52	1
:	2		73	0
:	8		55	0
	12		49	1
$i=m$	...			

$i = 1, \ldots, m$ ← training examples

$j = 1, \ldots, n$ ← features

target y is 0 or 1.

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$$

How to choose $\vec{w} = [w_1 \; w_2 \; \ldots \; w_n]$ and b ?

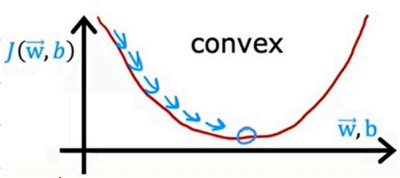
Squared error cost

$$J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m}\frac{1}{2}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)^2$$

only difference is that in linear regression they have put $\frac{1}{2}$ outside like $\frac{1}{2m}$

linear regression

$$f_{\vec{w},b}(\vec{x}) = \vec{w}\cdot\vec{x} + b$$



logistic regression

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$$



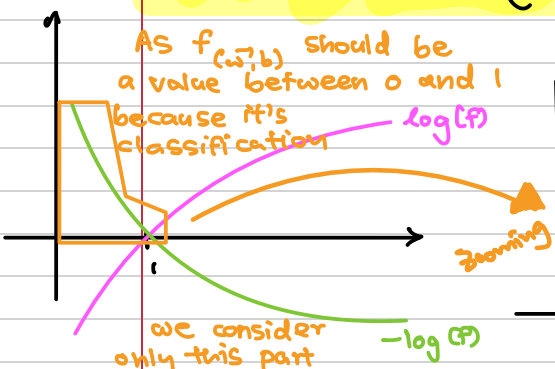
This is what happen if we use the same linear regression cost function.

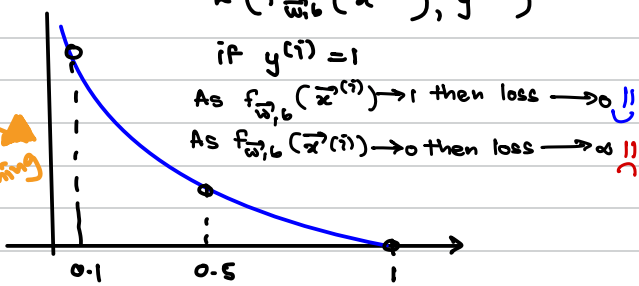
Because there are several local minima we cannot pick a one. Therefore for logistic regression squared error cost function is not the best way.

$$J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m} \boxed{\frac{1}{2}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)^2}$$

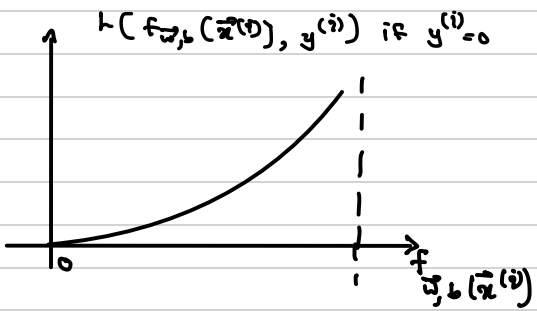
Loss $\rightarrow$ $L\left(f_{\vec{w},b}(x^{(i)}), y^{(i)}\right)$

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}, y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)}=1 \\ -\log\left(1-f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)}=0 \end{cases}$$









As $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 0$ then loss $\rightarrow 0$

As $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 1$ then loss $\rightarrow \infty$

$$J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m} \underbrace{L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right)}_{\text{Loss}}$$

$$= \begin{cases} -\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)}=1 \\ -\log\left(1-f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y=0 \end{cases}$$

can reach a global minimum (convex)

So we are taking the average for the dataset by taking all the losses. So we have to find w,b that minimize "J".

Simplified loss function

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)}=1 \\ -\log\left(1-f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)}=0 \end{cases}$$

$$L\left[f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = -y^{(i)}\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) - (1-y^{(i)})\log\left(1-f_{\vec{w},b}(\vec{x}^{(i)})\right)$$

if $y^{(i)}=1$:

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = -\log\left(f(\vec{x})\right)$$

if $y^{(i)}=0$:

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right) = -\log\left(1-f(\vec{x})\right)$$

cost function

$$J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m}\left[L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right)\right]$$

$$J(\vec{w},b) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)}\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) - (1-y^{(i)})\log\left(1-f_{\vec{w},b}(\vec{x}^{(i)})\right)$$

Training logistic regression

* Find $\vec{w}, b$
* Given new $\vec{x}$, output $f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$

$$P = (y=1|\vec{x};\vec{w},b)$$

Gradient descent

Cost function

$$J(\vec{w},b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{w},b}(x^{(i)})\right) + (1-y^{(i)})\log\left(1-f_{\vec{w},b}(x^{(i)})\right)\right]$$

repeat {

$$w_j = w_j - \alpha\frac{\partial}{\partial w_j}J(\vec{w},b) \qquad \frac{\partial}{\partial w_j}J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)x_j^{(i)}$$

$$b = b - \alpha\frac{\partial}{\partial b}J(\vec{w},b) \qquad \frac{\partial}{\partial b}J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)$$

}

- Simultaneous updates

If you think this looks exactly like linear regression $f_{\vec{w},b}(\vec{x})$ definition is different in this equation.

Linear regression $\quad f_{\vec{w},b}(\vec{x}) = \vec{w}\cdot\vec{x} + b$

Logistic regression $\quad f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$

Same concepts :-

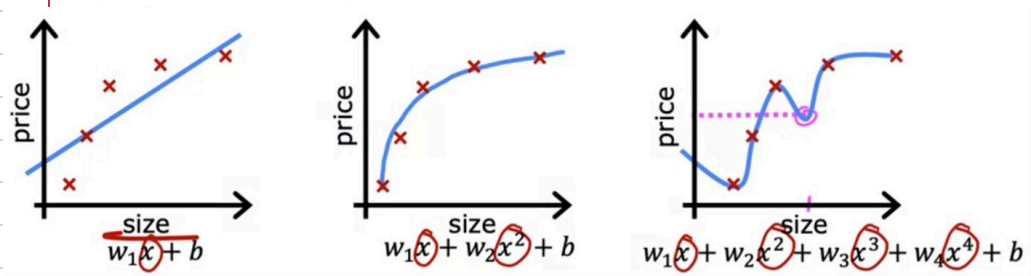
→ Monitor gradient descent (learning curve)

→ Vectorized implementation

→ Feature scaling

The problem of overfitting

~ Regression example ~



underfit
- Does not fit the training set well

Generalization
- Fits training set pretty well.

overfit
- Fits the training set extremely well.

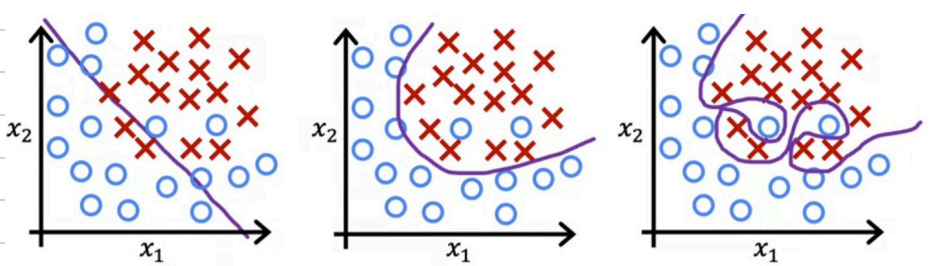
high bias

It is because it has few variables. So will not be able to match the target

high variance

Has too many variables. So passes through every point, and make it looks like its too well.

~ Classification



$z = w_1x_1 + w_2x_2 + b$

$f_{\vec{w},b}(\vec{x}) = g(z)$

g is the Sigmoid function

underfit high bias

$z = w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2 + b$

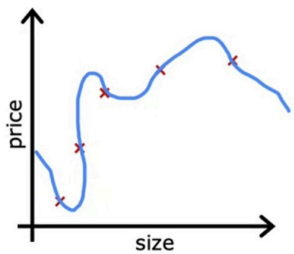
just right

$z = w_1x_1 + w_2x_2 + w_3x_1^2x_2 + w_4x_1^2x_2^2 + w_5x_1^2x_2^3 + w_6x_1^3x_2 + \ldots + b$

overfit high variance

~Addressing Overfitting~

① Collect more training examples



overfit



② Select features to include / exclude

In the previous examples our model features included the size x, as well as the size squared and this x squared and x cubed and x^4 and so on. These were lot of polynomial features.

Therefore, to reduce overfitting is to just not use so many of these polynomial features.

size	bedrooms	floors	age	avg income	...	distance to coffee shop	price
x_1	x_2	x_3	x_4	x_5		x_{100}	y

all features

+

insufficient data

↓

overfit

selected features

size
bedrooms
age
just right

feature selection

(more in course 2)

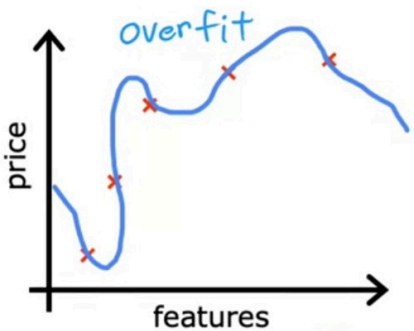
disadvantage

↓

useful features could be lost

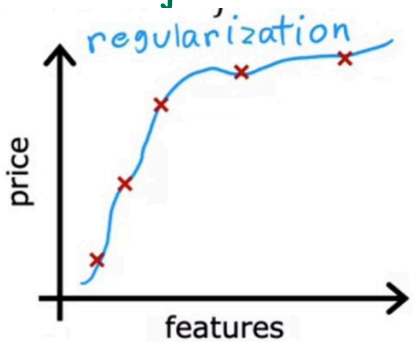
③. Regularization

Reduce the size of parameters w_j



$f(x) = 28x - 385x^2 + 39x^3 - 174x^4 + 100$

large values for w_j



$f(x) = 13x - 0.23x^2 + 0.000014x^3 - 0.0001x^4 + 10$

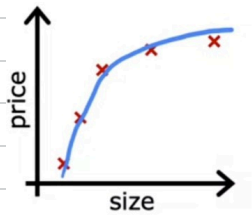
Smaller values for w_j

Parameter values are relatively large. If you were to eliminate the feature x^4 that corresponds to setting w_4 parameter to 0. So setting a parameter is equivalent to eliminating a feature, But Regularization is not doing something as harsh as eliminating it outright. It shrinks the parameter without it strictly equalizing to 0.

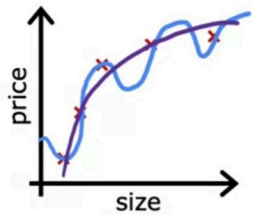
It doesn't make a huge difference whether you regularize the parameter b as well, you could do if you want or not if you don't. It's fine regularize the parameters w_1, w_2 all the way to w_n. If you really want to regularize b as well in practice, it should make very little difference.

~ Cost Function with regularization ~

Intuition



$w_1x + w_2x^2 + b$



$w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$

make w_3, w_4 really small (≈ 0)

$$\min_{\vec{w},b} \frac{1}{2m} \sum_{i=1}^{m} \left[f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right]^2 + 1000\,w_3^2 + 1000\,w_4^2$$

So minimizing this cost function is now minimizing the $1000\,w_3^2$ and $1000\,w_4^2$ and $\frac{1}{2m}\sum_{i=1}^{m}\left[f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right]^2$ is minimized.

As we have used a large co-efficient for w_3 and w_4 in cost function to minimize each term of w_j we need to choose very smaller values for w_j.

So let's build a model that uses 100 features.

In this situations, as we don't know which of these parameters are going to be the important ones, let's penalize all of them a bit and shrink all of them by adding this new term lambda (λ)

$$J(\vec{w},b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$$

n - no. of features

λ - regularization parameter ($\lambda > 0$)

- Instead of using $\lambda \sum_{j=1}^{n} w_j^2$, we also divide λ by $2m$ so that both the first and second terms are scaled by $\frac{1}{2m}$. By scaling both terms the same way it becomes easier to choose a good value for λ. Even if our training set size (m) growth, same λ will be used to work if we have this extra scaling by $2m$.

- We are not going to penalize the parameter b for being large. In practice it make very little difference whether you do it or not. And some machine learning engineers and actually some learning algorithm implementations will also include lambda over $2m$ times the b squared term.

$$\left[J(\vec{w},b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 \right] + \frac{\lambda}{2m} b^2$$

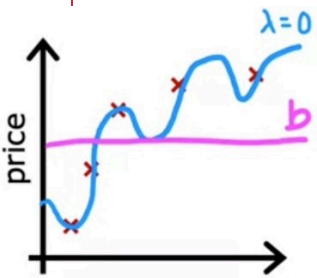
we use this cost function in this course

$$J(\vec{w},b) = \frac{1}{2m} \sum_{i=1}^{m} \underbrace{\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2}_{\text{mean squared error}} + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2}_{\text{Regularization term}}$$

mean squared error
fit data

Regularization term
keep w_j small

λ balances the both goals



λ balances both goals

choose $\lambda = 10^{10}$

$$f_{\vec{w},b}(\vec{x}) = \underset{\approx 0}{w_1 x} + \underset{\approx 0}{w_2 x^2} + \underset{\approx 0}{w_3 x^3} + \underset{\approx 0}{w_4 x^4} + b$$

$$f(x) = b$$

if λ is 0 the model will overfit. If λ is enormous like 10^{10}. This model will under fit. And so what you want is some value of λ that is in between that more appropriately balances these first and second terms of trading off, minimizing the mean squared error and keeping the parameters small.

~ Regularized Linear Regression ~

$$J(\vec{w},b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$$

$$\min_{\vec{w},b} J(\vec{w},b) = \min_{\vec{w},b}\left[\frac{1}{2m}\sum_{i=1}^{m}\left[f_{\vec{w},b}(\vec{x}^{(i)} - y^{(i)}\right]^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2\right]$$

Same as previous,

gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w},b)$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left[f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right]x_j^{(i)} + \frac{\lambda}{m} w_j$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w},b)$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left[f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right]$$

} simultaneous update

So previously we didn't regularize b, therefore it doesn't shrink and it remains same. whereas w_j changes because regularization term causes us to try to shrink w_j.

repeat {

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} \left[f_{\vec{w},b}(x^{(i)}) - y^{(i)} \right] x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$

$$b = b - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} \left[f_{\vec{w},b}(x^{(i)}) - y^{(i)} \right] \right]$$

} simultaneous update.

$$w_j = w_j - \frac{\alpha}{\lambda m} w_j - \alpha \left(\frac{1}{m} \sum_{i=1}^{m} \left[f_{\vec{w},b}(x^{(i)}) - y^{(i)} \right] x_j^{(i)} \right)$$

$$w_j = \left(1 - \frac{\alpha}{\lambda m} \right) w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left[f_{\vec{w},b}(x^{(i)}) - y^{(i)} \right] x_j^{(i)}$$

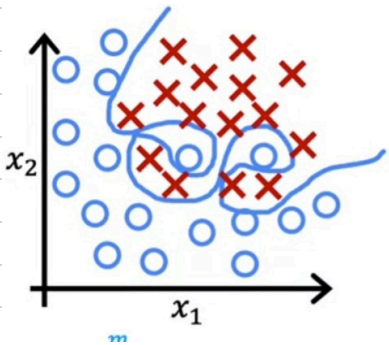
only co-efficient of w_j changes when comparing with previous gradient descent.

usual update

w_j is shrinked here. Previously it was 1. But now $0.01 \times \frac{2}{50} = 0.0002 \rightarrow (1 - 0.0002) = 0.9998$. It has shrinked slightly due to the regularization.

$(\alpha \times \frac{\lambda}{m})$

~ Regularized logistic regression ~



$$z = w_1x_1 + w_2x_2 + w_3x_1^2x_2 + w_4x_1^2x_2^2 + w_5x_1^2x_2^3 + \cdots + b$$

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-z}}$$

Cost function

$$J(\vec{w},b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{w},b}(x^{(i)})\right) + (1-y^{(i)})\log\left(1-f_{\vec{w},b}(x^{(i)})\right)\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$$

$$\min_{\vec{w},b} J(\vec{w},b) \longrightarrow w_j \downarrow$$

Same as previous,

gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w},b) \qquad \frac{1}{m}\sum_{i=1}^{m}\left[f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right]x_j^{(i)} + \frac{\lambda}{m}w_j$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w},b) \qquad \frac{1}{m}\sum_{i=1}^{m}\left[f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right]$$

} simultaneous update

same as linear regression only change is that here

$$f_{\vec{w},b}(x^{(i)}) = \frac{1}{1+e^{-\vec{w}}} = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$$